Template Document

Academic Support Agent

February 19, 2025

This policy memo discusses problems with the current U.S. patent system with a specific focus on software patents and recommends policies for action by the United States Patent and Trademark Office (USPTO). Although the USPTO does not have a definition of or collect information on software-related patents, since the granting of the first software patent [**?**] in 1968, software patents have occurred in a variety of technologies containing at least some element of software, covering things like sending messages or conducting business over the Internet (e.g., e-commerce) [**?**].

The groups with the greatest stake in the patentability, validity, and scope of software patents are innovators (e.g., big tech companies, startups, and individual developers), patent lawyers, and non-practicing entities (i.e., patent trolls who buy patents in order to profit by means of licensing or litigation instead of actually creating any new products or coming up with new ideas). Broadly speaking, however, all users of products containing (to-be-)patented software are affected by the current state of software patents. The remainder of the memo is organized as follows.

Section I reviews key cases and the law that shape how the USPTO treats software patents.

Section II identifies three major problems with software patents in the U.S.:

- The slow process in getting a software patent application reviewed and decided upon;
- The low quality of granted software patents in general; and
- The difficulty of challenging bad software patents efficiently.

Section III dives into three policy suggestions of varying degrees of radicalness for implementation at the USPTO:

- Roll out clearer guidelines for granting or rejecting software patents;
- Strengthen collaboration with the tech community, universities, and law schools; and
- Invest in an internal information technology (IT) system revamp in view of the growing power of artificial intelligence (AI).

Finally, Section IV concludes this memo with the limitations of the policies suggested and their implications beyond software patents.

1 Background: Key Cases and the Law

The purpose of this section is to lay the foundation for understanding the current landscape of the software patents and the USPTO s role in it by examining software patent-related judicial and legislative history.

Statutory Patentability. Patent examiners at the USPTO grant or reject patent applications based on the law. According to Sections 101-103 and 112 of the Patent Act, for any invention to be granted a patent (i.e., to be patentable), the subject matter must be eligible (i.e., it must be a process, machine, manufacture, or composition of matter, or improvement thereof) [**?**] and the invention must be novel [**?**], non-obvious [**?**], and well-disclosed [**?**].

Subject Matter Eligibility. It is unclear from the Patent Act whether and which software patents would be subject matter eligible, and patent examiners must determine subject matter eligibility in line with judicial precedents. In 1978, in the case of Parker v. Flook, the Supreme Court of the United States (SCOTUS) came close to banning software patents because software is just math, and an algorithm, or mathematical formula, is like a law of nature... a law of nature cannot be the subject of a patent. Flook, the patent applicant, argued that his patent was

more than just math because he had combined a computer program with a real-world industrial application, but the SCOTUS was unimpressed: The notion that post-solution activity, no matter how conventional and obvious in itself, can transform an unpatentable principle into a patentable process exalts form over substance. On the other hand, three years later, the SCOTUS approved a software-related patent on the idea of using a computer to manage a rubber-curing process, dynamically calculating the optimal time to open the press in order to produce perfectly cured rubber parts: Their process admittedly employs a well-known mathematical equation, but they do not seek to pre-empt the use of that equation. Given those rulings, it was almost impossible to patent software back then [**?**].

That situation changed in 1982, when Congress created a specialized appeals court for patent cases, the United States Court of Appeals for the Federal Circuit (CAFC). The CAFC loves software patents; the SCOTUS is more skeptical. In the next couple of decades, software patents like Amazon s infamous 1999 patent on the concept of shopping with one click [**?**] began to proliferate. In the landmark 2014 SCOTUS decision of CLS Bank v. Alice, the justices ruled unanimously that the patent—which claimed a software-based method of making sure two parties to a financial transaction keep their promises by having a third party hold their funds—was too abstract: The relevant question is whether the claims here do more than simply instruct the practitioner to implement the abstract idea of intermediated settlement on a generic computer. They do not. In the first three years after Alice, the CAFC rejected 92.3 percent of the patents challenged under the Alice precedent [**?**]. Now a patent claim under examination is rejected at the USPTO or the courts if it (1) contains an abstract idea (which is true for most software patents), and (2) does not add to it something extra that embodies an inventive concept (the two-step test or the Alice test).

Opposition Procedures. The USPTO currently offers several ways of challenging a patent s validity, but it was not always the case. It used to be the case that patents can only be invalidated by the courts [**?**], thereby allowing patent trolls to force their targets to pay hefty settlement demands in order to avoid the even heftier litigation fees. With the passage of the Leahy-Smith America Invents Act (AIA) in 2011, post-grant opposition procedures are expanded. In particular, the AIA expanded inter partes reexamination to become inter partes review (IPR) [**?**]. The IPR process allows the Patent Trial and Appeal Board (PTAB) to invalidate a patent, contributing to the sharp fall of cost of patent infringement litigation in general [**?**].

2 Problems with Software Patents

In this section I flesh out three major issues with software patents.

Slow Process. It takes nearly 2 years on average before a patent application is granted (if at all) [**?**], but software and software-related products usually have a life cycle as short as a few months to a couple of years. The situation is much better now than ten years ago, when the average pendency at the USPTO was over 3 years [**?**]. The USPTO currently has a backlog of over 540 thousand pending applications [**?**]. There is simply not enough patent examiners to review the patent applications, let alone carefully and thoroughly research previous inventions or publications, known as prior art. The slowness in getting a patent granted alone defeats one of the two main purposes of the patent system, namely, to reward and incentivize the inventors. As a result, in the interest of innovators, patent attorneys tend to draft software patent claims that are as wide-ranging as possible to protect the basic technical concept used in a string of future products or to cover potential improvements and innovations by their clients competitors.

Low Quality. Granted software patents often contain vague and overbroad claims, and the key ideas are usually obvious to a person having ordinary skill in the art to which the claimed invention pertains [**?**]. Even though patent examiners try very hard to find grounds to reject each patent, their time, energy, and tools are limited. The general low quality of software patents defeats the other of the two main purposes of the patent system, which is to disclose technical advances to the public [**?**]. We end up with software patents that are not useful either because the language is so vague and broad that a developer eager to learn the technical advances would have to tirelessly code and debug on their own before there is any viable product (since actual code is optional in a software patent), or because the implementation is so obvious that any programmer would be able to independently invent the same thing without knowledge of the patents. In addition, vague and overbroad patents prevent other people from patenting specific but truly original patents in a similar field, further stifling innovation [**?**].

Costly Opposition. If it is hard to ensure that granted patents are generally of high quality, it does not help if it is also hard to challenge the validity of bad patents. Introduction of procedures such as IPR made it easier to invalidate bad patents, but the USPTO s recent changes to how claims are interpreted in trial proceedings

(including IPR) before the PTAB made it harder again [**?**]. Instead of the broadest reasonable interpretation (BRI) standard previously used before the PTAB, ordinary and customary meaning is in place now instead. Consequently, challenged patents would be interpreted more narrowly and thus less likely to be invalidated over prior art. Even without the changes, the cost to file an IPR easily goes up to six digits, which is admittedly much cheaper than going to court but still extortionate for most small companies and individual developers who may seek to invalidate bad software patents owned by patent trolls or large companies so they could develop their own products [**?**].

3 Policy Proposals for the USPTO

In this section I propose three policies that the USPTO can implement to alleviate the problems above. All of them are based on past or ongoing initiatives at the USPTO. The goal of these policies is to help the patent system better serve its intended purposes of encouraging innovation and making useful information public.

Clearer Guidelines. [**?**] As can be seen from the convoluted judicial history of software patents and the tug-of-war between the SCOTUS and the CAFC, patent examiners and administrative law judges at the USPTO need clearer rules to guide them when they decide whether software patents meet the requirements of subject matter eligibility, among other things. In January 2019, the USPTO announced revised guidance for determining subject matter eligibility. Abstract ideas are now divided into concrete categories such as mathematical concepts, certain methods of organizing human activity, and mental processes. That helps eliminate some uncertainty in the application of the first step in the Alice test, namely, whether the patent claims are directed to an abstract idea. The guidance also explains that a patent claim or patent application claim that recites a judicial exception is not directed to the judicial exception if the judicial exception is integrated into a practical application of the judicial exception [**?**]. That aims to make the second step in the Alice test – whether the patent contains an inventive concept – easier to apply. The revised guidance is laudable progress in making the patent system more consistent and easier to navigate for staff at the USPTO, but I think we need clearer, more comprehensive guidelines that aim to cover issues related to software patents beyond subject matter eligibility for the staff as well as innovators and attorneys. Ideally, the guidelines will cover the recommended claim terms, what counts as enough disclosure, and what counts as non-obvious over prior art. As a consequence, there will be higher quality software patent applications and more efficient and predictable examinations and reviews. Implementation of this proposal would likely involve a committee reviewing areas of software patent-related law and policy that is unclear, drafting guidelines, and soliciting public feedback before rolling it out to the USPTO and the public. Although the USPTO does not have general substantive rulemaking power, its guidelines will likely have a significant impact on the landscape of software patents since any legislative effort takes a long time and the result is uncertain [**?**].

Stronger Collaboration. [**?**] More and more students are majoring in computer science, more and more people are getting involved in open-source projects, and more and more law schools are welcoming applicants with a technical background (by, for example, accepting the Graduate Record Examinations in addition to the Law School Admission Test). Now is the perfect time for the USPTO to consider untraditional ways of collaborating with the broad tech community, universities, and law schools to improve the overall quality of software patents and facilitate patent examination and review. For example, the USPTO could consider restarting and actively promoting its Peer-to-Patent program [**?**] across different schools and open-source communities so that interested people could submit potential prior art for patent examiners consideration. Previous pilot projects have had support from schools, technology partners, and corporate sponsors. To avoid potential conflict of interest, the USPTO could consider open-sourcing the entire Peer-to-Patent system, reducing the cost of the program and making it better known in the tech community. In addition, the USPTO could hire interested computer science majors and law students with technical backgrounds as interns to help review and shed light on software patent applications. If no budget can be diverted to hire additional interns with a tech background, the USPTO may ask universities and law schools to fund their students during the internship since what they do is beneficial to their studies and promotes public welfare.

Smarter System. Modernizing the IT system has been an ongoing effort at the USPTO [**?**] but with the advent of drastically improved performance of AI these years, perhaps AI may be incorporated into the patent examination and review system to help expedite various procedures and improve the consistency and predictability of decisions at the USPTO. AI would be especially useful in prior art search and may even be able to recommend decisions based on judicial precedents and past examiners decisions using machine learning techniques. Initial investment in AI technology may be costly, but the long-term benefits far outweigh the costs.

4 Conclusion: Limitations and Implications

What the USPTO can do is limited. A comprehensive and continuous software patent reform would also require joint efforts from Congress, the courts, and tech companies. Nonetheless, I hope this memo gives you additional inspiration. Beyond software patents, my proposal for a smarter system naturally affects the patent system in general, and those for clearer guidelines and stronger collaboration with the public can be easily adapted to other sectors. The proposals require significant time and resources to implement, but they all aim to bring an ideal patent system – one that stimulates innovation and promotes disclosure of inventions – closer to reality.